

An opinionated take on coding agents in computational science

Eoin Murray

Agenda

1. **Coding agents** — why now?
2. **Demolab v1** — ready for trial.
3. **Demolab v2** — autoresearch and lab scale.

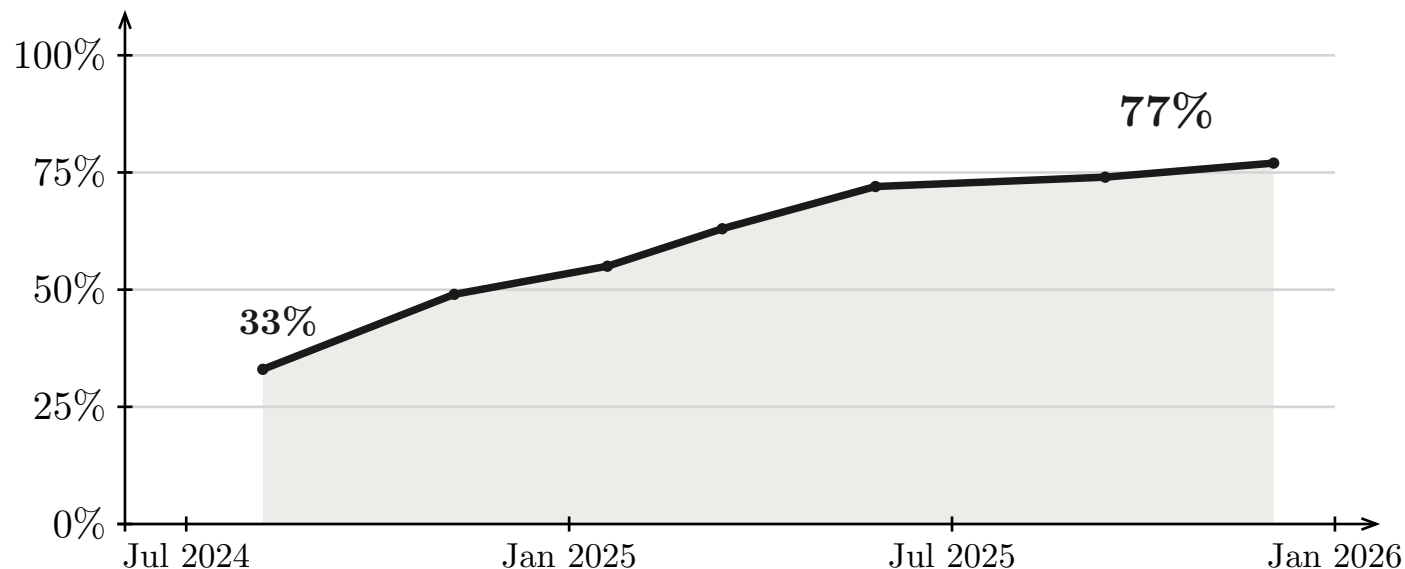
PART ONE

Coding agents

An observation

- **Software engineers** use coding agents like crazy.
- **Scientists**, far less so.

Coding agents are now useful



Share of real, human-validated GitHub issues an agent resolves end-to-end — SWE-bench Verified (swebench.com). Software bugs, not science.

autocomplete (2021) → chat (2023) → **agents that do real work (2025)**

What a coding agent is (demo)

- **Does the work**, not just autocomplete: reads, runs, edits, verifies.
- **Can do anything your terminal can do.**
- Many exist, commercial and open — no vendor lock-in.
- Only recently good enough — still uneven, still supervised.

Strengths of coding agents

- The same work faster.
- Do the things you always wanted but didn't have time.
- Autodocumentation — the work writes itself down as it goes.
- Read the literature — turn a paper straight into runnable code.

Weaknesses of coding agents

- **Confidently wrong** — fake APIs, numbers, citations.
- **Comprehension debt** — you ship code you don't understand.
- **Weak judgement** — won't kill a bad experiment.
- **Brainrot** — can't agent-code and read deeply the same day.

Powerful tool, needs supervision.

The price of coding agents

Agent	Entry	Heavy use/mo
GitHub Copilot	\$10/mo	\$39–100/mo
Cursor	\$20/mo	\$60–200/mo
Windsurf	\$20/mo	\$200/mo
Claude Code	\$20/mo	\$100–200/mo
Codex (ChatGPT)	\$20/mo	usage tail
Antigravity	\$20/mo	\$100–200/mo
DeepSeek (API)	API key	\$15–90/mo
Local Llama (8B)	\$0	not agent-grade

Bundled agents: seat + usage meter. DeepSeek: pay-per-token. Local Llama: free, assist-tier only.

PART TWO

Demolab v1 — a framework for the agent

What demolab is

- **Demolab writes code, runs programs, and documents results.**
- **A lab notebook for computational science** — the science lives as code.
- **One loop** — run → data → write-up → publish.
- **One build** — the whole repo becomes a website and PDFs.
- **Numbers come from the run** — never retyped, so the page can't drift.
- **Reproducible by default** — provenance stamped on every result.

Why demolab

- **Results drift** — retyped numbers, stale figures. Here they come from the run.
- **Reproducibility rots** — a year on, nothing builds. One command rebuilds it all.
- **Code and paper split up** — the experiment and the write-up live in one repo.
- **Agents need rails** — operate in plain language, keep the science yours.

Principles of demolab

1. **Tools compute, experiments analyse** — clean split.
2. **One experiment** = a runner + a write-up.
3. **Talk through files**, never imports.
4. **Raw data is disposable** — the record is committed.
5. **Bring any stack.**
6. **Provenance built in.**

The shape of a demolab repo (demo: pinglab)

<code>tools/</code>	the science – models & solvers
<code>experiments/</code>	the runners – <code>expNNN.py</code> per experiment
<code>writings/</code>	the writeups – one <code>.typ</code> per entry
<code>temp/</code>	the raw run data – gitignored, big
<code>artifacts/</code>	the distilled record – <code>data/</code> + <code>pdfs/</code> (committed)
<code>demolab.yaml</code>	branding + collections (optional)
<code>demolab-engine/</code>	the engine (black box you never edit)

Typst: a modern LaTeX

LaTeX

- Slow, multi-pass compiles.
- Cryptic error messages.
- PDF only.
- Macro soup and package hell.

Typst

- **Instant**, incremental compiles.
- **Readable** errors with line numbers.
- One source → **web and PDF**.
- **Imports files**, no hardcoded numbers.

The same beautiful math: selectable MathML on the web, typeset in the PDF.

Guides

```
my-coding-agent ▶ HELP █
```

```
GUIDES  # always on – the conventions
```

```
RULES HOUSESTYLE SLIDES STRUCTURE GLOSSARY SUPPORT
```

Type **HELP** — demolab walks you through its own documentation.

Runbooks

```
RUNBOOKS # on demand — run it, step by step
```

```
GETTING-STARTED TOUR LINT DOCTOR RED-TEAM STEELMAN
```

```
NEXT MIGRATE-CODE MIGRATE-STACK FROM-JUPYTER FROM-PAPER
```

```
EMBED-DOCS GROUND-CLAIMS UPDATE
```

Agent programs — read papers, convert code, migrate the stack,
update.

Driven by a handful of commands

<code>task install</code>	set up (Python deps via uv)
<code>task run -- exp000</code>	run an experiment end-to-end
<code>task dev</code>	serve the site, live-reload on save
<code>task build</code>	website + a PDF per entry + a book
<code>task test</code>	run the test suite

Or just operate manually — no agent required.

Use any stack you want

- **Default: Python** — `uv`, `.py` runners, `pytest`.
- **Files, not imports** — tools write `numbers.json`; runners never import tools.
- **Switch by asking** — migrate the stack to **MATLAB**; the agent runs a runbook.
- **MATLAB · R · Julia · Octave** — Typst and the contract stay put.
- **Python optional** — plotting only, or drop it entirely.

GETTING STARTED

demolab.eoinmurray.info

SUPPORT

github.com/eoinmurray/demolab

Open a GitHub issue and I'll fix it quickly.

PART THREE

Demolab v2 — the future of demolab

Autoresearch

- **Give the agent a goal and compute.**
- **It runs the loop** — goal → code → compute → read the data → document → iterate.
- **Every step is recorded** — a committed experiment and write-up per attempt.
- **The rails already exist** — provenance, reproducibility, and a published trail come free.

Lab scale

- **v1 is one person** — a single researcher's lab notebook.
- **Next: a lab on a GitHub org** — many users, one shared flow.
- **It compounds** — questions and answers, code review, shared code across the lab.